SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1718**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I D. Jeshurun Silas (192525324) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Jeshurun Silas

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.

i. What are the percept's for this agent? ii. Characterize the operating environment. iii. What are the actions the agent can take? iv. How can one evaluate the performance of the agent?

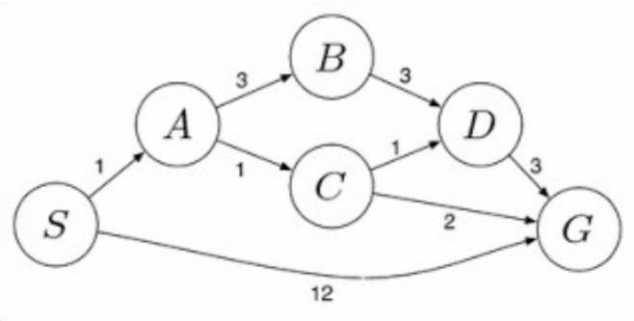
v. What sort of agent architecture do you think is most suitable for this agent? Why?

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations

Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Question 1: Water Jug Problem

1.1 Problem Understanding

Two jugs are available: a 4-gallon jug and a 3-gallon jug, neither with measuring marks. A pump can fill either jug. The goal is to have exactly **2 gallons in the 4-gallon jug**.

1.2 Problem Formulation (State-Space)

- **State:** an ordered pair (x, y), where x is the water in the 4-gallon jug ($0 \le x \le 4$) and y is the water in the 3-gallon jug ($0 \le y \le 3$).
- **Initial state:** (0, 0)
- **Goal state:** (2, n) for any n; here the target state is (2, 0).
- **Operators (production rules):**

No.	Rule	Effect
1	Fill the 4-gallon jug ($x < 4$)	$(x, y) \rightarrow (4, y)$
2	Fill the 3-gallon jug ($y < 3$)	$(x, y) \rightarrow (x, 3)$
3	Empty the 4-gallon jug on the ground ($x > 0$)	$(x, y) \rightarrow (0, y)$
4	Empty the 3-gallon jug on the ground ($y > 0$)	$(x, y) \rightarrow (x, 0)$
5	Pour 3-gallon into 4-gallon until 4-gallon is full ($x + y \ge 4, y > 0$)	$(x, y) \rightarrow (4, y - (4 - x))$
6	Pour 4-gallon into 3-gallon until 3-gallon is full ($x + y \ge 3, x > 0$)	$(x, y) \rightarrow (x - (3 - y), 3)$
7	Pour all of 3-gallon into 4-gallon ($x + y \le 4, y > 0$)	$(x, y) \rightarrow (x + y, 0)$
8	Pour all of 4-gallon into 3-gallon ($x + y \le 3, x > 0$)	$(x, y) \rightarrow (0, x + y)$

1.3 Solution

Using a breadth-first (shortest-path) exploration of the state space, the minimum solution needs **6 moves**. Two equally short solutions exist.

Solution A

Step	State (4-gal, 3-gal)	Action	Rule
0	(0, 0)	Start	–

Step	State (4-gal, 3-gal)	Action	Rule
1	(0, 3)	Fill the 3-gallon jug from the pump	2
2	(3, 0)	Pour the 3-gallon jug into the 4-gallon jug	7
3	(3, 3)	Fill the 3-gallon jug again	2
4	(4, 2)	Pour from the 3-gallon jug into the 4-gallon jug until it is full	5
5	(0, 2)	Empty the 4-gallon jug on the ground	3
6	**(2, 0)**	Pour the 2 gallons from the 3-gallon jug into the 4-gallon jug	7

Solution B

Step	State	Action	Rule
0	(0, 0)	Start	–
1	(4, 0)	Fill the 4-gallon jug	1
2	(1, 3)	Pour 4-gallon into 3gallon until full	6
3	(1, 0)	Empty the 3-gallon jug	4
4	(0, 1)	Pour the 1 gallon into the 3-gallon jug	8
5	(4, 1)	Fill the 4-gallon jug	1
6	(2, 3)	Pour 4-gallon into 3gallon until full	6
7	**(2, 0)**	Empty the 3-gallon jug (4-gallon holds 2)	4

(Solution B takes 7 moves, so Solution A is optimal.)

1.4 Interpretation

The 4-gallon jug holds exactly **2 gallons** after 6 moves in Solution A. This is possible because gcd(4, 3) = 1, so any whole quantity up to 4 gallons can be measured. Breadth-first search guarantees the shortest sequence because every move has equal cost.

Question 2: Mars Rover as an Intelligent Agent

The Mars Rover is a robotic agent that navigates the Martian surface, collects soil and rock samples, analyses them with onboard instruments and transmits the results to Earth. Its task environment is described below using the PEAS framework.

i. Percepts

Percepts are what the rover's sensors deliver:

- Camera images (navigation cameras, panoramic and hazard cameras) showing terrain, rocks, slopes and obstacles
- Spectrometer readings and chemical or mineral composition data of rocks and soil
- Wheel encoders, inertial and tilt sensors (position, speed, orientation, wheel slip)
- Temperature, dust and atmospheric readings
- Battery and solar-panel power levels, internal system health data
- Commands and updates received from the Earth control centre **ii. Characterisation of the Operating Environment**

Property	Classification	Reason
Observability	Partially observable	Sensors see only the nearby terrain; hidden rocks, craters and soil below the surface are unknown
Agents	Single agent	Only the rover acts (ground control only sends high-level commands)
Determinism	Stochastic	Wheel slip, sand, dust storms and sensor noise make outcomes uncertain
Episodic / Sequential	Sequential	Current moves affect power, position and future sampling
Static / Dynamic	Dynamic	Dust storms, light and temperature change while the rover is deliberating
Discrete / Continuous	Continuous	Position, speed, temperature

Property	Classification	Reason
		and time are continuous values
Known / Unknown	Largely unknown	The terrain is new and only partly mapped

The environment is therefore **partially observable, single-agent, stochastic, sequential, dynamic, continuous and unknown**, which is among the hardest environment types for an agent.

iii. Actions

- Move forward or backward, turn, climb or stop; avoid obstacles and re-plan routes
- Deploy the robotic arm and drill, scoop or collect soil and rock samples
- Operate the on-board instruments to analyse a sample
- Take photographs and scan the surroundings
- Orient and deploy the solar panels or antenna, and transmit data to Earth
- Enter safe/sleep mode to conserve power or survive dust storms, and recharge **iv.**

Performance Measure

The rover's performance can be evaluated by:

- Number and scientific value of samples collected and analysed
- Area of terrain explored and number of target sites reached
- Accuracy and completeness of the data transmitted to Earth
- Safety: no collisions, tip-overs or getting stuck
- Efficient use of energy, time and mission lifetime
- Reliability and fault recovery (system health maintained)

v. Most Suitable Agent Architecture

A **utility-based, goal-driven agent with a model-based reflex layer** (a hybrid, layered architecture) is the most suitable:

- **Model-based reflex layer:** the environment is only partially observable, so the rover must maintain an internal map and state (position, power, terrain). Time-critical reactions such as stopping before a cliff or tilting limit are handled by fast reflexes.
- **Goal-based layer:** the rover must achieve mission goals (reach a site, collect a sample), which needs planning and search over action sequences.
- **Utility-based layer:** several goals compete (science value vs. power, risk and time), so a utility function is needed to trade off and choose the best action.
- **Learning component (optional):** improves terrain classification and navigation from experience in an unknown environment.

A simple reflex agent is insufficient because it cannot remember the past or plan, and a purely goalbased agent cannot weigh risk against reward.

Question 3: 8-Queens Problem

3.1 Problem Understanding

Place 8 queens on an 8 × 8 chessboard so that no two queens attack each other, i.e. no two share the same row, column or diagonal. The configuration in the figure is not a solution because the queen in the first column and the queen in the last column lie on the same diagonal (a diagonal conflict).

3.2 Problem Formulation

Two formulations are possible.

Complete-state formulation: begin with all 8 queens on the board and move them to reduce conflicts (suited to local search).

Incremental formulation (used here for systematic search): start with an empty board and add one queen at a time.

Component	Definition
States	Any arrangement of 0 to 8 queens on the board such that no queen attacks another. Queens are placed in the leftmost empty column
Initial state	Empty board (no queens)
Actions	Add a queen in a safe square of the leftmost empty column
Transition model	Returns the board with the queen added to the specified square
Goal test	8 queens are on the board and none is attacked
Path cost	Not relevant (only the final arrangement matters; each placement may be given cost 1)

State-space size. If we simply allow any 8 queens on any 64 squares, there are about $64 \times 63 \times \ldots \times 57 \approx 1.8 \times 10^{14}$ sequences. Placing one queen per column, never in an attacked square, reduces this to only **2,057 states**, making the problem tractable.

3.3 Exploring the Search Space

- **Uninformed strategies:** Breadth-first search would work but stores a huge number of partial boards. **Depth-first search with backtracking** is the better choice: the solution depth is fixed at 8, memory use is small (one path), and a dead end (no safe square in a column) simply triggers backtracking to the previous column.
- **Branching:** at depth 0 there are 8 choices; the number of safe choices shrinks as queens are added, which prunes the tree.

- **Pruning rule:** two queens at (r_1, c_1) and (r_2, c_2) attack each other if $r_1 = r_2$, or $|r_1 - r_2| = |c_1 - c_2|$. The diagonal conflict in the given figure (first and last column) would be caught by this test.

3.4 Backtracking Procedure

```
PLACE(col):
  if col == 8: return SOLUTION for
  row in 1..8:
    if SAFE(row, col):     # no same row or diagonal with earlier queens board[col]
      = row if PLACE(col + 1) succeeds: return SUCCESS board[col] = none  #
      backtrack
  return FAILURE
```

3.5 A Valid Solution

Row numbers for columns 1 to 8: **(1, 5, 8, 6, 3, 7, 2, 4)**.

- All rows are different.
- Values of (row − column): 0, 3, 5, 2, −2, 1, −5, −4 are all distinct, so no shared diagonal.
- Values of (row + column): 2, 7, 11, 10, 8, 13, 9, 12 are all distinct, so no shared anti-diagonal.

The 8-queens problem has **92 solutions** (12 unique ones when rotations and reflections are ignored).

Question 4: OLA Cab Booking Problem

4.1 Type of Agent

A **goal-based problem-solving agent** is suitable, with a **utility-based element** for choosing the cab type.

- **Goal:** reach the destination from the pickup location.
- **Why goal-based:** the agent formulates a goal, builds a problem (states, actions, costs) and searches for a sequence of actions that reaches the goal.
- **Why utility-based element:** several cab types (Mini, Micro, Sedan, Shared, Prime, etc.) all achieve the goal, but differ in cost, comfort, waiting time and travel time. A utility function ranks them according to the customer's preference.

4.2 Problem Formulation

Component	Definition
Initial state	Customer at pickup location, no cab booked
Actions	Enter the destination, select a cab type, confirm booking, track the cab, ride, pay
Transition model	Booking confirmed → cab assigned → cab arrives → trip in progress → destination reached
Goal test	Customer is at the destination

Path cost	Fare + waiting time + travel time (weighted by customer comfort preference)

4.3 Pseudocode

```
function OLA-PROBLEM-SOLVING-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty) state (description of current
  world state) goal (initially null) problem (problem formulation)

  state <- UPDATE-STATE(state, percept)  # location, drivers nearby, prices, traffic if seq is
  empty then
    goal          <- FORMULATE-GOAL(state)     # reach destination D from pickup P
    problem <- FORMULATE-PROBLEM(state, goal) seq   <- SEARCH(problem) if seq
    = failure then return a null action
  action <- FIRST(seq) seq
        <- REST(seq)
  return action

function SEARCH(problem) available <- GET-AVAILABLE-CABS(pickup, destination) # Mini, Micro,
  Sedan, Shared, Prime ...
  if available is empty then return failure best <- null; bestUtility <- -infinity for each cab in
  available do u <- UTILITY(cab.fare, cab.eta, cab.travelTime, cab.comfort, customer.preference)
  if u > bestUtility then best <- cab; bestUtility <- u return [SELECT(best), CONFIRM-
  BOOKING, TRACK-CAB, BOARD, RIDE, PAY, ARRIVE]
```

Execution flow: the agent perceives the situation, formulates the goal, searches the available cab options, picks the cab with the highest utility, executes the action sequence one step at a time, and replans if a driver cancels or the percepts change.

Question 5: Uniform Cost Search (UCS)

5.1 Problem Understanding

Find the least-cost path from the start warehouse **S** to the destination warehouse **G** in the weighted directed graph. Edges read from the figure:

Edge	Cost
S → A	1
S → G	12
A → B	3

A → C	1
B → D	3
Edge	Cost
C → D	1
C → G	2
D → G	3

5.2 Method Selection

UCS is chosen because edge costs differ (fuel + time) and the objective is the minimum **total path cost**. UCS always expands the frontier node with the lowest path cost g(n) using a priority queue, so it is complete and optimal for non-negative costs. (Breadth-first search would minimise the number of edges, not cost, and would wrongly favour the direct S → G edge of cost 12.)

5.3 Solution Process

Step	Node expanded (cost g)	Frontier after expansion (node : cost, via)
1	S (0)	A : 1 (S), G : 12 (S)
2	A (1)	C : 2 (A), B : 4 (A), G : 12
3	C (2)	D : 3 (C), B : 4, G : 4 (C) *(replaces 12)*,
4	D (3)	B : 4, G : 4 (C); path to G via D costs 6, not better, so ignored
5	B (4)	G : 4 (C); D via B costs 7, not better, ignored
6	**G (4)**	Goal selected for expansion, so search stops

Costs of the main paths to G:

- S → A → C → G = 1 + 1 + 2 = **4**
- S → A → C → D → G = 1 + 1 + 1 + 3 = 6
- S → A → B → D → G = 1 + 3 + 3 + 3 = 10
- S → G = 12

(If B and G tie at cost 4, either may be popped first; the final answer is the same.)

5.4 Final Answer

- **Order of expansion:** S, A, C, D, B, G
- **Optimal path: S → A → C → G**
- **Minimum total cost: 4 units** (fuel + time)

5.5 Interpretation

The cheapest delivery route passes through warehouses A and C and costs 4 cost units, which is 8 units cheaper than the direct route S → G (12). The direct route is the shortest in number of edges but the most expensive. UCS is optimal because it delays accepting G until the lowest-cost path to it is the cheapest item in the frontier. The trade-off is that UCS may expand many low-cost nodes in large networks, so its time and memory grow with the number of nodes cheaper than the optimal solution.

Conclusion

This assessment applied core AI concepts from CO1: state-space search for the water jug problem, PEAS analysis and agent design for the Mars Rover, problem formulation with backtracking for 8-Queens, a goal- and utility-based problem-solving agent for the OLA cab application, and Uniform Cost Search for least-cost warehouse routing. *Prepared by D. Jeshurun Silas*